

8. Game Theory and Sequences

Contents

Mex Calculator	1
Remove Game	1
K-th Balanced Bracket Sequence	2
Next Balanced Sequence	2

Mex Calculator

```
1 class mex_calculator {
2     map<int, int> count;
3     set<int> missing;
4     public:
5     mex_calculator(const vector<int>& arr, int upper_bound) { // n log n
6         for (int x : arr)
7             count[x]++;
8         for (int i = 0; i <= upper_bound + 1; ++i)
9             if (count[i] == 0)
10                missing.insert(i);
11    }
12    void insert(int x) { // log
13        count[x]++;
14        if (count[x] == 1)
15            missing.erase(x);
16    }
17    void remove(int x) { // log
18        if (count[x] == 0) return;
19        count[x]--;
20        if (count[x] == 0)
21            missing.insert(x);
22    }
23    int get_mex() { // 1
24        return *missing.begin();
25    }
26 };
```

Remove Game

```
1 void solve() {
2     int n; cin >> n;
3     vector<int> v(n);
4     getv(v);
```

```

5     ll diff[n+1][n+1]{}; // mx diff p1-p2 on interval [l, r]
6     for (int l = n-1; ~l; l--) {
7         for (int r = 0; r < n; r++) {
8             // dp[l][r] = mx( v[l]-dp[l+1][r], v[r]-dp[l][r-1] )
9             if (l == r)
10                diff[l][r] = v[l];
11            else diff[l][r] = max<ll>(v[l] - diff[l+1][r], v[r]-diff[l][r-1]);
12        }
13    }
14
15    cout << (accumulate(all(v), 0ll) + diff[0][n-1])/2;
16 }

```

K-th Balanced Bracket Sequence

```

1 //O(n^2)
2 string kth_balanced(int n, int k) {
3     vector<vector<int>> d(2*n+1, vector<int>(n+1, 0));
4     d[0][0] = 1;
5     for (int i = 1; i <= 2*n; i++) {
6         d[i][0] = d[i-1][1];
7         for (int j = 1; j < n; j++)
8             d[i][j] = d[i-1][j-1] + d[i-1][j+1];
9         d[i][n] = d[i-1][n-1];
10    }
11    string ans;
12    int depth = 0;
13    for (int i = 0; i < 2*n; i++) {
14        if (depth + 1 <= n && d[2*n-i-1][depth+1] >= k)
15            {
16                ans += '(';
17                depth++;
18            } else {
19                ans += ')';
20                if (depth + 1 <= n)
21                    k -= d[2*n-i-1][depth+1];
22                depth--;
23            }
24    }
25    return ans;
26 }

```

Next Balanced Sequence

```

1 bool next_balanced_sequence(string & s) { // O(n)
2     int n = s.size();
3     int depth = 0;
4     for (int i = n - 1; i >= 0; i--) {
5         if (s[i] == '(')
6             depth--;
7         else
8             depth++;
9         if (s[i] == '(' && depth > 0) {
10             depth--;

```

```
11         int open = (n - i - 1 - depth) / 2;
12         int close = n - i - 1 - open;
13         string next = s.substr(0, i) + ')' +
14         string(open, '(') + string(close, ')');
15         s.swap(next);
16         return true;
17     }
18 }
19 return false;
20 }
```